IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RÉPUBLIQUE
FRANÇAISE
Liberté
Égalité
Fraternité

Quantum Generative Adversarial Networks Hybrid Architectures and Convergence Analysis

Leveraging Variational Quantum Circuits for High-Dimensional Data Synthesis

Antony Manuel

IMT Nord Europe
Department of Computer Science
Master of Data Science

Lille, April 2026

Abstract

This report presents the design, implementation, and iterative refinement of a hybrid quantum-classical generative adversarial network for image synthesis, following the Patch-Quantum Wasserstein GAN (PQWGAN) architecture trained with a gradient-penalty objective (WGAN-GP) from Tsang et al.'s work [1]. The generator is composed of 16 independent parameterized quantum circuit (PQC) subgenerators, each implemented in PennyLane with a 5-qubit ansatz (4 data qubits plus 1 ancilla) and a layered structure of trainable single-qubit rotations interleaved with nearest-neighbor entangling gates. Each subgenerator produces a 4x4 pixel patch via ancilla-conditioned measurement probabilities, which are assembled into full 16x16 grayscale images. The discriminator is a classical convolutional (or fully-connected) critic network trained under the WGAN-GP objective, augmented with a total variation regularization term to encourage spatial coherence across independently generated patches. The system is evaluated on a binarized two-class subset of MNIST downsampled to 16x16 resolution.

The report documents these findings, the corrective patches applied, and concrete recommendations for further improving training stability, reproducibility, and quantitative evaluation of the resulting quantum GAN.

Keywords: Quantum computing, Parameterized Quantum Circuits, Generative Adversarial Networks, Quantum Machine Learning, Quantum Generative Adversarial Networks, Training Stability

Notation

$\mathcal{X}$	Set.
$\mathbb{N}$	Set of integers.
$\mathbb{R}$	Set of real numbers.
$\subset$	Strict inclusion.
$\subseteq$	Inclusion with possible equality.
$\lvert \cdot \rvert$	Absolute value of a real number.
$\lVert \cdot \rVert$	Euclidean norm of a vector.
x	A real scalar belonging to $\mathbb{R}$.
$x \in \mathbb{R}^n$	A vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A matrix in $\mathbb{R}^{n \times d}$.
I_n	Identity matrix of size n.
$X^\top$	Transpose of X.
X^{-1}	Inverse of X.
x	A random scalar variable.
$x \in \mathbb{R}^n$	A random vector in $\mathbb{R}^n$.
$X \in \mathbb{R}^{n \times d}$	A random matrix in $\mathbb{R}^{n \times d}$.
$P(x)$, $P(\mathbf{x})$	Probability distribution of x (resp. joint probability distribution of $\mathbf{x}$).
$x \sim P_x$	Random variable x follows a probability distribution P_x.
$\mathbb{E}(x)$	Expectation of x with respect to $P(x)$.
$\mathrm{Var}(x)$	Variance of x under $P(x)$.
x_i	i-th element of the vector x.
$X_{i,j}$	Element at row i, column j of the matrix X.
$X_{i,:}$	i-th row of the matrix X.
$X_{:,i}$	i-th column of the matrix X.
$f \circ g$	Composition of functions f and g.
dy/dx	Derivative of y with respect to x.
$\partial y/\partial x$	Partial derivative of y with respect to x.
$\partial f/\partial x$	Jacobian matrix $J \in \mathbb{R}^{n \times p}$ of $f : \mathbb{R}^n \to \mathbb{R}^p$.

Sometimes we use a function f whose argument is a scalar but apply it to a vector $f(x)$ or a matrix $f(X)$. This denotes the element-wise application of f to the array.

$$* \ * \ *$$

Contents

Contents **3**

1 Introduction **5**

2 Background **7**
1 Generative Modeling . . . 7
1.1 Generative Adversarial Networks . . . 7
1.2 Wasserstein Generative Adversarial Networks . . . 8
2 Quantum computing . . . 9
2.1 Quantum Bits . . . 9
2.2 Quantum gates and circuits . . . 10
2.2.1 Single Qubits Gates . . . 10
2.2.2 Multi Qubit Gates . . . 11
2.3 Variational Quantum Algorithms . . . 12

3 QGAN Framework **13**
1 Related Work . . . 13
2 Architecture overview . . . 13
2.1 Generator logic . . . 13
2.2 Critic structure . . . 13
3 Experiments and Training Strategy . . . 14
3.1 Training objective . . . 14
3.2 Training algorithm . . . 15
3.3 Dataset and libraries . . . 15
3.4 Hyperparameters . . . 15

4 Results **16**
1 Results Analysis . . . 16
1.1 Generators parameters . . . 16
1.1.1 Generator learning rate . . . 16
1.1.2 Gradient penalty . . . 16

1.1.3 Total variation 16
1.1.4 Generator layers 16
2 Discussion 16

5 Conclusion **18**

Bibliography **19**

Annexes **20**

A Appendix **21**
1 Details on generator hyperparameters effects. 21

B Annexe **23**
1 Cost Functions 23
1.1 Critic Loss with Gradient Penalty 23
1.2 Generator Loss with Total Variation Regularization 23
2 Measurement and Quantum Expectation 23
2.1 Expectation Values and Probability Distributions 24
2.2 Patch-Based Probability Slicing 24
3 Optimizers 24
3.1 Adam Update Equations 24
4 Gradient Descent and Quantum Differentiation 24
4.1 Parameter-Shift Rule 25

Introduction

Over the past decade, generative models have emerged as one of the most dynamic areas of development within deep learning, continually finding novel applications across diverse domains, including image synthesis, musical composition, and drug discovery. Spanning from Variational Autoencoders (VAEs) to autoregressive models, the field of generative modeling has yielded a plethora of architectures and techniques. Each paradigm possesses distinct strengths and limitations, collectively pushing the boundaries of machine-driven creation. Among these varied approaches, Generative Adversarial Networks (GANs), introduced in 2014 [2], have stood out for their remarkable capacity to generate highly realistic data. GANs have been successfully deployed across a wide spectrum of applications, such as image synthesis [2], text generation, molecular and physical data simulation, and 3D object generation. However, the deployment of classical GANs is constrained by several significant challenges, notably mode collapse, vanishing gradients, extreme sensitivity to hyperparameter selection, and an immense demand for both data and computational resources. Numerous architectural and algorithmic enhancements have been proposed to mitigate these issues [3], for instance Wasserstein GANs [4] employ a distinct distance metric to formulate a loss function that improves training stability, while Karras et al. [5] proposed to incrementally scales the resolution of generated images during the training phase. Despite these advancements, training classical GANs remains inherently complex and computationally expensive, frequently necessitating exhaustive empirical tuning to achieve optimal convergence.

Concurrently, quantum computing has emerged as a transformative computational paradigm, holding the promise of solving certain classes of problems that are intractable for classical computers, such as Shor's algorithm for prime factorization and Grover's algorithm for unstructured database search. Driven by these theoretical advantages, there has been a burgeoning interest in exploring the intersection of quantum computing and machine learning. This synthesis has catalyzed the development of Quantum Machine Learning (QML) algorithms, which aim to harness quantum speedups for complex pattern recognition and optimization tasks. Within the QML landscape, extending quantum methodologies to generative modeling presents a particularly compelling frontier, culminating in the conceptualization of Quantum Generative Adversarial Networks (QGANs) [6, 7]. By replacing traditional neural networks with parameterized quantum circuits (PQCs)

[8] within the generator, the discriminator, or both, QGANs theoretically possess greater representational capacity. This quantum advantage could enable the learning of highly complex, high-dimensional probability distributions more efficiently than classical frameworks, potentially alleviating the severe computational bottlenecks inherent to traditional GAN training.

In this work, we consider the following research papers [1, 9, 10] which propose novel QGAN architectures and training methodologies that aim to enhance the stability and performance of quantum generative models.

Background

1 Generative Modeling

In machine learning, generative modeling is a type of unsupervised learning focused on learning the underlying distribution of a dataset to generate new samples that resemble the original data. Generative models can be broadly categorized into explicit and implicit frameworks. Explicit generative models aim to define or approximate the explicit probability distribution of the data, allowing for likelihood evaluation and direct sampling; examples include Variational Autoencoders (VAEs) [11] and diffusion models [12]. Conversely, implicit generative models, such as Generative Adversarial Networks (GANs) [2], learn to generate realistic samples through adversarial training without explicitly modeling the underlying probability density function. These models span a wide range of domains, including image synthesis, text generation, and data augmentation, and are frequently utilized in financial environments, healthcare, and molecular biology. Because GANs are the primary focus of this research, the following sections provide a detailed overview of their architecture, training process, and subsequent architectural improvements.

1.1 Generative Adversarial Networks

Introduced by Goodfellows et al.[2], Generative Adversarial Networks are models able to learn the distributed probabilities underlying training data and generate new data based on that probability. Moreover GANs are really usefull for understanding distribution probability in unlabeled data, and generate data that are not present in the initial training set.

Basically, a GAN consists of two models, a generator G that outputs synthetic samples $\hat{x}$ given a noise variable input z, bringing potential diversity in the generating process, and is trained to capture the real data distribution in order to generate samples as close as possible to train data. The second model is a discriminator D that estimates the probability of a given sample x being synthetic or coming from the real dataset.

This system inspired from the game theory implies two scenario. The first one emphasizes the goal of the discriminator to classify the input x as real or fake by minimizing

the difference between its output probability $D(x) = P(x|y)$ and the true label y. The second scenario appears when noise inputs z to the generator are randomly sampled from the model's prior over the latent variables, then the discriminator receives input $G(z) = \hat{x}$, a fake sample created by the generator, then the discriminator strives to make $D(\hat{x})$ approach 0, while the generator strives to make the same quantity approach 1.

At the end, if both models have sufficient abilities, then the game reached the Nash equilibrium, in other words, it's corresponding to P_G, the learned distribution matching P_{data} the real then D having a probability $1/2$ of correctly classifying real and fake samples, which means G is able to produce samples indistinguishable from the real data, and D is unable to differentiate between them.

The loss of the model is based on two components, the generator loss and the discriminator loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(\hat{x}))] \tag{2.1}$$

1.2 Wasserstein Generative Adversarial Networks

As previously mentioned, training classical GANs is notoriously difficult, frequently suffering from mode collapse, vanishing gradients, and extreme sensitivity to hyperparameter configurations. To mitigate these pathologies, Arjovsky et al. [4] proposed the Wasserstein GAN (WGAN), which substitutes the traditional Jensen-Shannon divergence with the Wasserstein distance. To formalize this approach while ensuring training stability, Gulrajani et al. [13] introduced the WGAN with Gradient Penalty (WGAN-GP). Under this framework, the Discriminator no longer classifies inputs with a probability $p \in [0, 1]$; instead, it acts as a *critic* D that outputs a scalar score representing the *realness* of the data. Therefore, the objective function of WGAN-GP is formulated as:

$$\min_G \max_{D\in\mathcal{D}} V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z\sim p_z}[D(G(z))] - \lambda \mathbb{E}_{\hat{x}\sim p_{\hat{x}}}\left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2\right] \tag{2.2}$$

where the first two terms compute the original Wasserstein distance, and the third term represents the gradient penalty weighted by the hyperparameter λ. The penalty is evaluated on interpolated samples $\hat{x} = \epsilon x + (1-\epsilon)G(z)$, where $\epsilon \sim \mathcal{U}[0, 1]$. This penalty enforces a 1-Lipschitz constraint on the Critic by penalizing deviations of the gradient norm from 1, effectively preventing the gradients from vanishing or exploding.

Incorporating the WGAN-GP framework is particularly critical when designing a patch-based Quantum GAN (QGAN) for generating structural data like 28×28 MNIST digits. First, the Lipschitz constraint regularizes the critic's landscape, preventing its gradients from becoming too steep, which directly mitigates the severe issue of barren plateaus (narrow gradients) inherent to Parameterized Quantum Circuits (PQCs). Second, since a patch-based architecture generates independent sub-regions of the image, a standard

GAN loss often struggles with spatial coherence, resulting in a disjointed, mosaic-like final output. By providing a continuous real-valued score instead of a binary classification, the WGAN-GP critic offers smoother and more informative gradient feedback. This guides the quantum generator to effectively learn global spatial correlations across boundaries, ensuring seamless integration of the patches.

2 Quantum computing

This section provides a brief overview of the fundamental concepts of quantum computing. It begins with an introduction to quantum information and the representation of quantum states, followed by the description of quantum gates and circuits. Finally, it introduces the concept of Variational Quantum Algorithms (VQAs).

2.1 Quantum Bits

In quantum physics, each quantum state ψ of each quantum system can be represented by a ket $|\psi\rangle$, up to a global phase $e^{i\varphi}$, which is a vector of an Hilbert space $\mathcal{H}$ with its norm equal to 1.

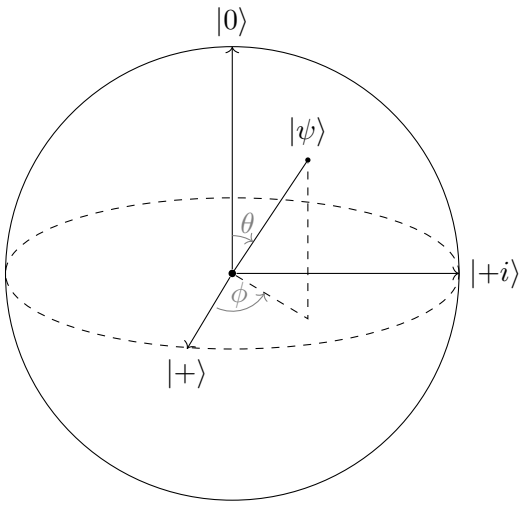


Fig. 2.1: Bloch Sphere

Unlike a classical bit, which is restricted to a binary state $b \in \{0, 1\}$, a qubit exists as a unit vector in a two-dimensional complex Hilbert space $\mathcal{H} \cong \mathbb{C}^2$. Using the computational basis $\mathcal{B} = \{|0\rangle, |1\rangle\}$, a general state $|\psi\rangle$ is expressed as a linear combination:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{2.3}$$

where $\alpha, \beta \in \mathbb{C}$ are probability amplitudes. According to the Born rule, the probability of measuring the state in $|0\rangle$ is $|\alpha|^2$ and in $|1\rangle$ is $|\beta|^2$, requiring the normalization condition $|\alpha|^2 + |\beta|^2 = 1$. Let's write the amplitudes in polar form, such as $\alpha = ae^{i\varphi}$ and $\beta = be^{i\gamma}$, where a, $b \in \mathbb{R}$. Hence, we can rewrite $|\psi\rangle$ as

$$|\psi\rangle = e^{i\varphi}(a\,|0\rangle + be^{(\gamma-\varphi)}\,|1\rangle) \tag{2.4}$$

since the global phase, $e^{i\varphi}$, does not affect the outcome measurement probabilities, and that 2.4 describe the exact same state :

$$|\psi\rangle = a\,|0\rangle + be^{\phi}\,|1\rangle \tag{2.5}$$

Additionally, the normalization of $|\psi\rangle$ requires that $a^2 + b^2 = 1$ in the way, we can parameterize the state using two real angles θ and ϕ:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle \tag{2.6}$$

where $\theta \in [0, \pi]$ represents the latitude and $\phi \in [0, 2\pi)$ the longitude. As a result, the state of a single qubit can be visualized as a vector in the Bloch sphere 2.1.

2.2 Quantum gates and circuits

In quantum computing, the manipulation of qubit states is executed through quantum circuits, i.e., computational models consisting of a sequence of quantum gates applied to a register of qubits. Mathematically, these gates are represented by unitary operators U (where $U^\dagger U = I$), a property that preserves the norm of the quantum state, thereby ensuring that the transformation is completely reversible and the total probability strictly remains equal to 1.

2.2.1 Single Qubits Gates

As the qubits evolve within the circuit, their state transformation mathematically corresponds to multiplying the state vector by the unitary matrix. Several gates are commonly used, such as the Pauli gates (X, Y, Z) and the Hadamard gate (H):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Consequently, the transition from an initial state $|\psi_0\rangle$ to a final state $|\psi_f\rangle$ is governed by the equation $|\psi_f\rangle = U\,|\psi_0\rangle$, where U denotes the unitary operator corresponding to the sequence of gates applied within the circuit.

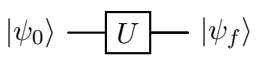


Fig. 2.2: Representation of a gate U applied to a state $|\psi_0\rangle$ resulting in a state $|\psi_f\rangle$.

2.2.2 Multi Qubit Gates

While single-qubit gates enable transformations on individual qubits, multi-qubit gates are fundamental for creating entanglement and establishing quantum correlations across composite quantum systems. The state space of an n-qubit system is described by the tensor product of the individual Hilbert spaces, $\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \cdots \otimes \mathcal{H}_n \cong (\mathbb{C}^2)^{\otimes n}$, of dimension 2^n.

Multi-qubit operations are represented by $2^n \times 2^n$ unitary matrices acting on the composite state. The most prominent two-qubit gate is the Controlled-NOT (CNOT) gate (or CX gate). It acts on a control qubit and a target qubit, flipping the state of the target qubit (applying a Pauli-X operation) if and only if the control qubit is in the state $|1\rangle$:

$$\text{CNOT} = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{2.7}$$

When applied to a superposition state, such as $\frac{1}{\sqrt{2}}(|0\rangle+|1\rangle)\otimes|0\rangle$, the CNOT gate generates the maximally entangled Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \tag{2.8}$$

Other commonly utilized two-qubit operations include the Controlled-Z (CZ) gate, the SWAP gate, and various parameterized controlled rotations.
The Controlled-Z (CZ) gate applies a phase flip to the target qubit if the control qubit is in the state $|1\rangle$, represented by the matrix:

$$\text{CZ} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \tag{2.9}$$

The Swap gate exchanges the quantum states of two qubits:

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{2.10}$$

Additionally, Controlled Phase (CR_ϕ) and Controlled Rotation gates implement continuous, parameterized multi-qubit interactions conditioned on the state of the control register.

In the context of Parameterized Quantum Circuits (PQCs) and Quantum Generative Adversarial Networks (QGANs), multi-qubit entangling gates are arranged in structured patterns (e.g., circular, nearest-neighbor, or all-to-all topologies) between parameterized rotation layers. These entangling layers enable the circuit to learn and express complex, high-dimensional non-local probability distributions that cannot be factored into independent marginal distributions.

2.3 Variational Quantum Algorithms

In addition to these basic gates, parameterized rotation gates $R_x(\theta)$, $R_y(\theta)$, and $R_z(\theta)$ enable arbitrary rotations on the Bloch sphere, as depicted in Fig. 2.3.

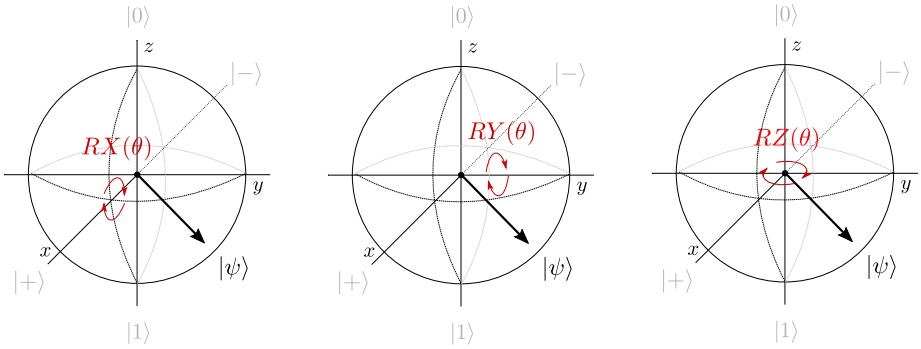


Fig. 2.3: Rotation gates on the Bloch sphere.

Note. Illustration from Pennylane codebook more reference here.

Therefore, a Variational Quantum Circuit (VQC), also known as an ansatz, incorporates these parameterized gates. The underlying parameters are optimized using classical optimization algorithms to minimize a cost function. This hybrid quantum-classical approach, through Variational Quantum Algorithms (VQAs)[14], allows us to leverage the computational advantages of quantum circuits while utilizing classical resources for optimization, which represents a cornerstone approach in quantum machine learning and quantum optimization.

In the context of a QGAN, the generator typically relies on such parameterized quantum circuits. It applies rotation gates to steer the state vector across the Bloch sphere, aiming to align the output with the target data distribution. The ultimate objective is to optimize these parameters so that the resulting quantum measurement statistics become indistinguishable from the real data distribution provided to the discriminator.

QGAN Framework

1 Related Work

2 Architecture overview

Based on the WGAN-GP framwork [4, 13] which demonstrates the effectivness of GAN training improvements. Moreover, recent work on quantum generative adversarial networks [1, 9] opted for a similar set up as in WGAN-GP, but with a quantum generator and a classical critic[10]. In the following section, we provide an overview of the architecture of the generator and the critic.

2.1 Generator logic

The generator is composed of S so-called sub-generators, each of which is a variation quantum circuit. It is important to replicate the non-linearity in the generator, but a classic quantum circuit is linear, so we need to introduce non-linearity in the generator. Several ansatzes have been proposed in the literature, and we will focus on the one proposed by [10] and the one proposed by [1], which propose different sources of non-linearity.

However, both ansatzes follow a similar strucutre, which consists of N qubits split into A ancilla qubits and D data qubits, and L layers of parameterized quantum gates. The latent vector z is first encoded using R_y gates, then the VQC is applied to produce the output. At the end, we measure the qubits to obtain the generated output.

2.2 Critic structure

Three structures are proposed in this work for the critic, one is a classical neural network as used in WGAN-GP [13], an other one is a classical convolution neural network, and the last one is a quantum circuit. However, the learning process with the quantum critic is more challengin due to the barren landscape problem [15], which makes the training of the quantum critic more difficult.

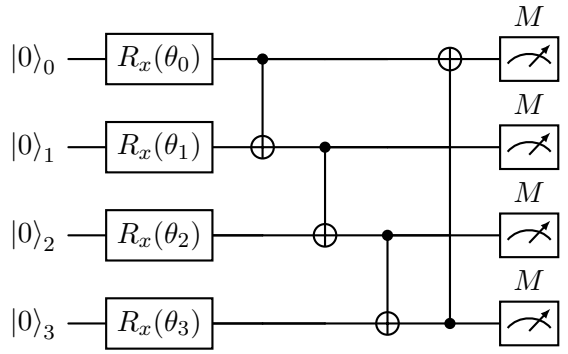


Fig. 3.1: Basic Entangling Ansatz circuit for 4 qubits. Here, θ_i represents the parameter `params[0][i]`, and M represents the observable.

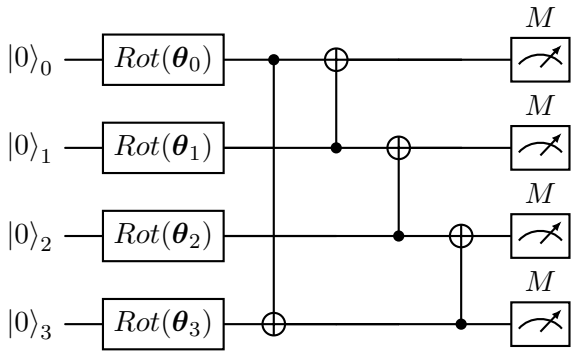


Fig. 3.2: Strongly Entangling Ansatz circuit for 4 qubits. $\boldsymbol{\theta}_i$ represents the three Euler angles for wire i (`params[0][i]`), and M represents the observable.

3 Experiments and Training Strategy

After defining the architecure of the generator and the critic, we will now discuss the training strategy, including the objective function, the training algorithm, and the experimental setup.

3.1 Training objective

The training objective of the Patch-Quantum Wasserstein GAN (PQWGAN) is formulated under the Wasserstein GAN with Gradient Penalty (WGAN-GP) framework, augmented with a Total Variation (TV) regularization term to promote spatial coherence across independently generated sub-patches.

Critic Objective The classical critic D is trained to maximize the Wasserstein distance while enforcing a 1-Lipschitz continuity constraint via a gradient penalty. The loss function minimized by the critic parameters is defined as:

$$\mathcal{L}_D = \mathbb{E}_{z\sim p_z}\left[D(G(z))\right] - \mathbb{E}_{x\sim p_{\text{data}}}\left[D(x)\right] + \lambda_{\text{gp}}\,\mathbb{E}_{\hat{x}\sim p_{\hat{x}}}\left[\left(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1\right)^2\right], \tag{3.1}$$

where $\hat{x} = \epsilon x + (1-\epsilon)G(z)$ with $\epsilon \sim \mathcal{U}[0,1]$ represents points sampled along straight lines between real data samples x and generated samples $G(z)$, and $\lambda_{\text{gp}} = 10.0$ is the gradient penalty coefficient.

Generator Objective The quantum generator G aims to maximize the critic's score for generated samples (or equivalently, minimize its negative expectation) while minimizing boundary discontinuities across adjacent patches via total variation regularization:

$$\mathcal{L}_G = -\mathbb{E}_{z\sim p_z}\left[D(G(z))\right] + \lambda_{\text{tv}}\,\mathcal{L}_{\text{tv}}(G(z)), \tag{3.2}$$

where $\lambda_{\text{tv}} = 0.1$ denotes the total variation regularization coefficient, and $\mathcal{L}_{\text{tv}}$ penalizes spatial variance across adjacent pixels in the assembled image:

$$\mathcal{L}_{\text{tv}}(X) = \sum_{i,j}\left(|X_{i+1,j} - X_{i,j}| + |X_{i,j+1} - X_{i,j}|\right). \tag{3.3}$$

3.2 Training algorithm

3.3 Dataset and libraries

Experimentations are conducted on the MNIST and Fashion-MNIST datasets, which consist of 28x28 grayscale images of handwritten digits and fashion items, respectively. The implementation of the quantum circuits and training algorithms is carried out on Python3, using the PennyLane library[16], which provides a high-level interface for quantum machine learning and supports various quantum hardware backends, and the PyTorch library[17] for classical optimization and data handling.

3.4 Hyperparameters

The learning rate η_r is set to 0.005 for the generator and 0.0002 for the critic, with the Adam optimizer used for both networks and β_1, β_2 set to 0.0 and 0.9 respectively for both generator and critic. Additionally, the number of sub-generators S is set to 16, with each sub-generator number of qubits N set to 5, and the number of layers L set to 2. Regarding the gradient penalty coefficient λ_{gp}, it is set to 10.0, while the total variation loss coefficient λ_{tv} is set to 0.1.

Results

1 Results Analysis

1.1 Generators parameters

1.1.1 Generator learning rate

1.1.2 Gradient penalty

1.1.3 Total variation

1.1.4 Generator layers

2 Discussion

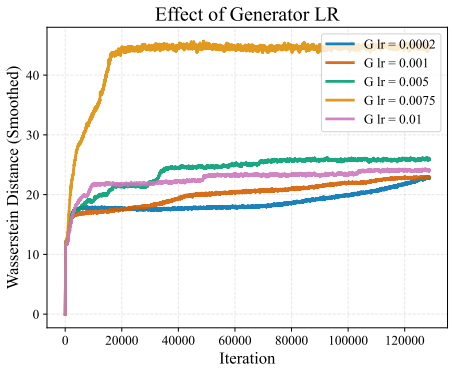


Fig. 4.1: Learning rate of the generator during training.

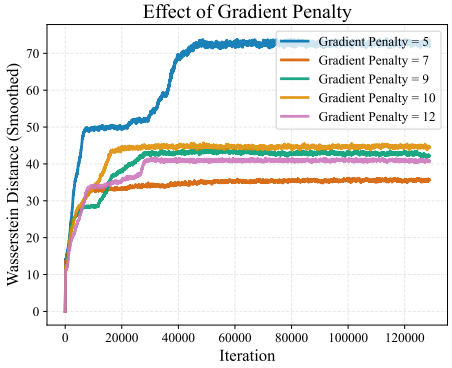


Fig. 4.2: Gradient penalty of the generator during training.

Conclusion

Bibliography

[1] S. L. Tsang, M. T. West, S. M. Erfani, and M. Usman, "Hybrid quantumclassical generative adversarial network for high-resolution image generation," *IEEE Transactions on Quantum Engineering*, vol. 4, p. 119, 2023.

[2] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," 2014.

[3] L. Mescheder, A. Geiger, and S. Nowozin, "Which training methods for gans do actually converge?," 2018.

[4] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein gan," 2017.

[5] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive growing of gans for improved quality, stability, and variation," 2018.

[6] S. Lloyd and C. Weedbrook, "Quantum generative adversarial learning," *Physical Review Letters*, vol. 121, July 2018.

[7] P.-L. Dallaire-Demers and N. Killoran, "Quantum generative adversarial networks," *Physical Review A*, vol. 98, July 2018.

[8] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, "Parameterized quantum circuits as machine learning models," *Quantum Science and Technology*, vol. 4, p. 043001, Nov. 2019.

[9] J. Jäger, F. J. Kiwit, and C. A. Riofrío, "Scaling quantum machine learning without tricks: High-resolution and diverse image generation," 2026.

[10] H.-L. Huang, Y. Du, M. Gong, Y. Zhao, Y. Wu, C. Wang, S. Li, F. Liang, J. Lin, Y. Xu, R. Yang, T. Liu, M.-H. Hsieh, H. Deng, H. Rong, C.-Z. Peng, C.-Y. Lu, Y.-A. Chen, D. Tao, X. Zhu, and J.-W. Pan, "Experimental quantum generative adversarial networks for image generation," *Physical Review Applied*, vol. 16, Aug. 2021.

[11] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2022.

[12] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," 2015.

[13] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of wasserstein gans," 2017.

[14] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational quantum algorithms," *Nature Reviews Physics*, vol. 3, p. 625644, Aug. 2021.

[15] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, "Barren plateaus in quantum neural network training landscapes," *Nature Communications*, vol. 9, Nov. 2018.

[16] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi, J. M. Arrazola, U. Azad, S. Banning, C. Blank, T. R. Bromley, B. A. Cordier, J. Ceroni, A. Delgado, O. D. Matteo, A. Dusko, T. Garg, D. Guala, A. Hayes, R. Hill, A. Ijaz, T. Isacsson, D. Ittah, S. Jahangiri, P. Jain, E. Jiang, A. Khandelwal, K. Kottmann, R. A. Lang, C. Lee, T. Loke, A. Lowe, K. McKiernan, J. J. Meyer, J. A. Montañez-Barrera, R. Moyard, Z. Niu, L. J. O'Riordan, S. Oud, A. Panigrahi, C.-Y. Park, D. Polatajko, N. Quesada, C. Roberts, N. Sá, I. Schoch, B. Shi, S. Shu, S. Sim, A. Singh, I. Strandberg, J. Soni, A. Száva, S. Thabet, R. A. Vargas-Hernández, T. Vincent, N. Vitucci, M. Weber, D. Wierichs, R. Wiersema, M. Willmann, V. Wong, S. Zhang, and N. Killoran, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," 2022.

[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," 2019.

Appendix

1 Details on generator hyperparameters effects.

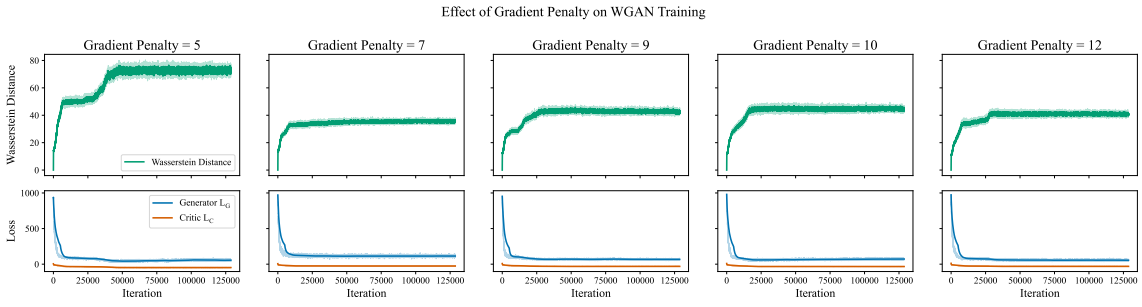


Fig. A.1: Gradient penalty effect on the training process.

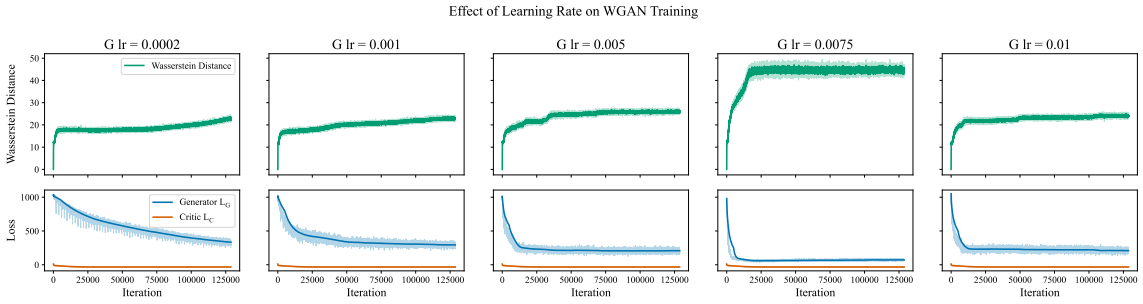


Fig. A.2: Learning rate of the generator during training.

Annexe

1 Cost Functions

In hybrid quantum-classical generative adversarial architectures, the objective functions guide both the quantum generator $G_{\boldsymbol{\theta}}$ and the classical critic $D_{\boldsymbol{\phi}}$ to achieve convergence towards a Wasserstein-1 equilibrium[cite: 1].

1.1 Critic Loss with Gradient Penalty

The critic loss implements the Kantorovich-Rubinstein duality of the Wasserstein metric augmented with a gradient penalty to enforce the 1-Lipschitz condition:

$$\mathcal{L}_D(\boldsymbol{\phi}) = \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}}\left[D_{\boldsymbol{\phi}}(G_{\boldsymbol{\theta}}(\boldsymbol{z}))\right] - \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}\left[D_{\boldsymbol{\phi}}(\boldsymbol{x})\right] + \lambda_{\text{gp}}\mathbb{E}_{\hat{\boldsymbol{x}}\sim p_{\hat{\boldsymbol{x}}}}\left[\left(\|\nabla_{\hat{\boldsymbol{x}}} D_{\boldsymbol{\phi}}(\hat{\boldsymbol{x}})\|_2 - 1\right)^2\right], \quad \text{(B.1)}$$

where $\hat{\boldsymbol{x}} = \epsilon\boldsymbol{x} + (1-\epsilon)G_{\boldsymbol{\theta}}(\boldsymbol{z})$ with $\epsilon \sim \mathcal{U}[0,1]$ defines the linear interpolation distribution $p_{\hat{\boldsymbol{x}}}$, and $\lambda_{\text{gp}} = 10.0$ is the regularization parameter.

1.2 Generator Loss with Total Variation Regularization

The quantum generator minimizes the critic's realness assessment while applying a Total Variation (TV) penalty to mitigate mosaic boundary artifacts across independently generated sub-patches[cite: 1]:

$$\mathcal{L}_G(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}}\left[D_{\boldsymbol{\phi}}(G_{\boldsymbol{\theta}}(\boldsymbol{z}))\right] + \lambda_{\text{tv}}\mathcal{L}_{\text{tv}}(G_{\boldsymbol{\theta}}(\boldsymbol{z})), \quad \text{(B.2)}$$

where $\lambda_{\text{tv}} = 0.1$, and the anisotropic Total Variation regularizer for a 2D image $X \in \mathbb{R}^{H\times W}$ is given by:

$$\mathcal{L}_{\text{tv}}(X) = \frac{1}{H\times W}\sum_{i=1}^{H-1}\sum_{j=1}^{W-1}\left(|X_{i+1,j} - X_{i,j}| + |X_{i,j+1} - X_{i,j}|\right). \quad \text{(B.3)}$$

2 Measurement and Quantum Expectation

In Parameterized Quantum Circuits (PQCs), classical data outputs are extracted by measuring observables on the parameterized quantum state $|\psi(\boldsymbol{\theta}, \boldsymbol{z})\rangle$.

2.1 Expectation Values and Probability Distributions

Given an initial state $|0\rangle^{\otimes N}$, the state vector is evolved under a parameterized unitary operator $U(\boldsymbol{\theta}, \boldsymbol{z})$:

$$|\psi(\boldsymbol{\theta}, \boldsymbol{z})\rangle = U(\boldsymbol{\theta}, \boldsymbol{z})|0\rangle^{\otimes N}. \tag{B.4}$$

For an observable $\hat{\mathcal{M}}$, the expected measurement value is calculated as:

$$\langle \hat{\mathcal{M}} \rangle_{\boldsymbol{\theta},\boldsymbol{z}} = \langle \psi(\boldsymbol{\theta}, \boldsymbol{z})|\hat{\mathcal{M}}|\psi(\boldsymbol{\theta}, \boldsymbol{z})\rangle = \mathrm{Tr}\left(\hat{\mathcal{M}}|\psi(\boldsymbol{\theta}, \boldsymbol{z})\rangle\langle\psi(\boldsymbol{\theta}, \boldsymbol{z})|\right). \tag{B.5}$$

2.2 Patch-Based Probability Slicing

For patch generation, computational basis projective measurements $\Pi_k = |k\rangle\langle k|$ are applied on data qubits conditioned on the ancilla qubit state $|q_a = 0\rangle$[cite: 1]:

$$p(k \,|\, q_a = 0) = \frac{\langle\psi(\boldsymbol{\theta}, \boldsymbol{z})| \left(|0\rangle\langle 0|_a \otimes |k\rangle\langle k|_d\right) |\psi(\boldsymbol{\theta}, \boldsymbol{z})\rangle}{\langle\psi(\boldsymbol{\theta}, \boldsymbol{z})| \left(|0\rangle\langle 0|_a \otimes \mathbb{I}_d\right) |\psi(\boldsymbol{\theta}, \boldsymbol{z})\rangle}, \quad k \in \{0, 1, \ldots, 2^D - 1\}, \tag{B.6}$$

where $D = 4$ data qubits yield a 16-dimensional vector corresponding to a flattened 4×4 pixel patch[cite: 1].

3 Optimizers

Both the generator and critic parameters are updated using the Adaptive Moment Estimation (Adam) optimizer. Adam tracks exponentially decaying averages of past gradients (m_t) and squared gradients (v_t) to compute adaptive step sizes.

3.1 Adam Update Equations

At optimization step t with objective gradient $g_t = \nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}_t)$:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{B.7}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{B.8}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \tag{B.9}$$

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon_{\text{stab}}} \hat{m}_t, \tag{B.10}$$

where hyperparameter settings for this architecture are $\beta_1 = 0.0$, $\beta_2 = 0.9$, $\eta_G = 0.005$, and $\eta_D = 0.0002$.

4 Gradient Descent and Quantum Differentiation

Optimizing variational quantum circuits via classical gradient descent requires computing gradients of quantum expectation values with respect to circuit parameters $\boldsymbol{\theta}$.

4.1 Parameter-Shift Rule

Because quantum state intermediate superpositions cannot be directly accessed without state collapse, analytical gradients for gates generated by Pauli operators $G = e^{-i\frac{\theta}{2}P}$ (where $P^2 = \mathbb{I}$) are evaluated using the parameter-shift rule:

$$\frac{\partial\langle\hat{\mathcal{M}}\rangle}{\partial\theta_i} = \frac{\langle\hat{\mathcal{M}}\rangle\left(\theta_i + \frac{\pi}{2}\right) - \langle\hat{\mathcal{M}}\rangle\left(\theta_i - \frac{\pi}{2}\right)}{2}. \tag{B.11}$$

This allows exact computation of the quantum generator gradients on quantum hardware or simulators without relying on finite-difference numerical approximations.